COMP3011 Coursework 1

Evan's Super Cool Baseball Data API

Evan Madurai (ID: 201634521)

13 March 2026

Key links

GitHub Repo · Frontend · API · MCP Server · Hosted Docs · PDF Docs · Presentation Slides

---

Project Motivation

For my coursework, I have implemented *Evan's Super Cool Baseball Data API*, a Web Service for retrieving, searching, and saving modern statistical baseball data as well as creating and competing with fantasy baseball teams.

This project was inspired by my love for the sport of Baseball (*Ok Blue Jays!*) and the field of Sabermetrics.

Data Sources

All data for this project was sourced from the Lahman Baseball Database, originally created by Sean Lahman and currently maintained by Bryan Walko at the Society for American Baseball Research (SABR).

The data was downloaded in CSV format, and a seed script (*prisma/seed.ts*) was created with the assistance of Artificial Intelligence for parsing and transforming the data into a relational format suitable for this project.

The dataset is licenced under *Creative Commons Attribution-ShareAlike 3.0 Unported (CC BY-SA 3.0) license*.

Technology Stack

Programming language choice

The first technical decision I made was that of programming language. I chose to use Typescript due to it's robust and powerful type system as well as massive package library. This was also a great learning opportunity to me, as it was my first time writing server-side Typescript code. Other programming languages such as Go and Rust were considered that would have likely had better performance, but would have had longer development time implications.

Web Framework

I chose to implement this project using NestJS. This decision was driven by the robust TypeScript ecosystem, its modular architecture, and the wealth of available documentation. Express and Fastify were also considered.

In particular, NestJS encourages a highly structured API design, with the application divided into Modules, Controllers, and Services. This architectural pattern facilitates a strict separation of concerns: **Controllers** handle incoming HTTP requests and routing, while **Services** encapsulate the complex business logic and data persistence [1].

By leveraging NestJS's built-in Dependency Injection system, each component remains loosely coupled, allowing for a modular application design where individual units can be tested in isolation using mocks. Furthermore, this structure ensures the codebase remains scalable and maintainable [2].

Database

This project uses a PostgreSQL instance as its persistent datastore. The choice of relational database is an often overthought decision. Other relational databases (MySQL, Microsoft SQL Server, SQLite) were considered, but PostgreSQL was chosen due to developer familiarity and its vast extension library.

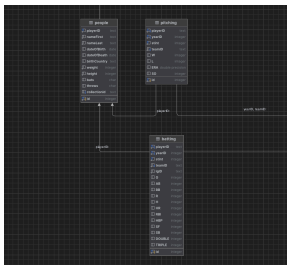


Figure 1: Player, batting, pitching relation

Instead of interfacing directly with SQL queries, the decision was made to use an ORM. This provides type-safe queries, reducing errors [3]. Prisma was chosen due to its intuitive syntax [4].

CI/CD (Continuous Integration/Continuous Deployment)

Effective CI/CD (Continuous Integration/Continuous Deployment) pipelines ensure that code changes are automatically built, tested, and validated before being merged or deployed. This aims to catch issues before deployment, increasing uptime and decreasing potential security issues for end users [5].

This application has three CI/CD pipelines, all deployed on GitHub Actions. They are as follows:

- Linting: Ensures code complies with the project style guide, performs static type-checking, and identifies potential anti-patterns before they enter the codebase.
- Unit Tests: All unit tests must pass before code can be merged.
- E2E Tests: Using Testcontainers in a DinD (Docker in Docker) environment to test behavior against a real non-production database.

All three pipelines must pass with no issues before code changes can be merged into the main branch of the repository. Once code changes have been merged into main, the application is automatically deployed to production.

Other development tools used

- Git, GitHub, Postman, Docker, Render, Winston, Jest

Documentation

Documentation for the API is generated automatically using Swagger. The documentation is closely coupled to the application code, using the NestJS/Swagger to inline document endpoints and resources. This ensures documentation stays up to date with any code changes.

Swagger generates a webpage with detailed API documentation, this can be found at comp3011.evanmadurai.co.uk/docs. It can also be found in raw YAML format at comp3011.evanmadurai.co.uk/docs-yaml. This is useful for either generating client libraries (with OpenAPI-generator) or for feeding into LLMs for potential client programming assistance.

API Design

Architectural Style

The API follows REST (Representational State Transfer) conventions, organising resources around nouns (`/player`, `/teams`, `/collection`, `/match`) and using HTTP verbs to express intent [6]. All responses are returned as JSON, and standard HTTP status codes are used throughout — `200` for successful reads, `201` for resource creation, `400` for validation errors, `401` for authentication failures, `404` for missing resources, and `409` for conflicts such as duplicate usernames.

Paginated list endpoints follow a consistent envelope format, returning a `data` array alongside a `meta` object containing `total`, `page`, `limit`, `totalPages`, `hasNextPage`, and `hasPreviousPage` fields. This design makes client-side pagination straightforward and predictable across all list endpoints.

JWT Authentication & RBAC

This application implements JWT-based authentication, where users register or log in via `/auth/register` or `/auth/login` and receive a signed token that must be supplied as a `Bearer` token on protected routes. Passwords are stored with bcrypt salted hashing.

Beyond simple authentication, the application implements Role-Based Access Control (RBAC) with two roles: *user* and *admin*. Read endpoints for players and teams are public, while all mutation operations (create, update, delete) are restricted to admin users. This is enforced via a NestJS Guard that inspects the JWT payload on each request, returning `401 Unauthorized` for missing or invalid tokens and implicitly `403 Forbidden` for insufficient role.

This separates concerns cleanly — a public consumer of the API can browse players, teams, and analytics without authenticating, while the integrity of the dataset is protected from unauthorised modification.

Resource Overview

Players (`/player`) represent the core data resource. The full set of CRUD operations is supported: players can be listed with optional `name` and `birthCountry` filters, retrieved individually by ID, created with batting and pitching season records, updated via `PATCH`, and deleted. Player responses include computed career statistics — batting average, OBP, slugging percentage, career pitching totals, and single-season highs — derived from the underlying season-level data rather than stored directly.

Teams (`/teams`) expose historical season records. Teams are identified by a standarised franchise code (e.g. `BOS`, `NYA`) and a year, and can be queried across all seasons for a given franchise or retrieved for a specific season. Teams support filtering by league and year, and a single season record can be deleted. Team responses include batting and pitching aggregates as well as season results (division win, World Series win, etc.).

Collections (`/collection`) are user-owned lists of players, analogous to fantasy baseball rosters. Authenticated users can create, read, update, and delete their own collections. Each collection holds between 1 and 50 player IDs. Collections are the foundation of the match system.

Matches (`/match`) allow two collections to compete against one another. A match is created by submitting two collection IDs (each with 9 players), and a result is recorded. Match history is scoped to the authenticated user, making a personal league table.

Analytics (`/analytics/league-leaders`) provides a read-only ranked endpoint for batting and pitching statistics. Clients specify a `category` (batting or pitching) and a `stat` (e.g. `homeRuns`, `era`, `battingAverage`), with optional `year` and `league` filters. This endpoint demonstrates the analytical capability of the dataset beyond simple CRUD.

Validation

Input validation is handled at the DTO (Data Transfer Object) layer using NestJS's built-in validation pipeline [7]. Where validation fails, the API returns a structured `400` response with an `errors` array detailing which fields failed and why.

Advanced features

Computed player statistics

A dedicated service computes advanced second-order statistics for players and teams that are not present in the raw Lahman dataset. These are derived at query time from the underlying season-level records rather than stored, ensuring they remain consistent with any data updates.

These stats were inspired by sabermetrics research and include Batting Average (BA), On-Base Percentage (OBP), Slugging Percentage (SLG), and On-Base Plus Slugging (OPS) — a composite metric widely regarded as one of the best single-number summaries of a batter's offensive value [8]. Career highs for home runs and hits are also surfaced, computed as the maximum across all season records for a given player.

Testcontainers for E2E testing

Instead of simply mocking the database interactions, I chose to setup TestContainers. This creates a real database instance for queries to interact with. This allows deeper-level testing of system functionality, increasing reliability on code changes.

These E2E tests are also configured to run in CI/CD, which is a more advanced Docker-in-Docker setup.

Frontend

A basic web frontend was developed alongside the API, serving as a proof-of-concept for how the endpoints could be composed into a full application. Rather than being a core deliverable, it demonstrates the API's practical utility — showing that the authentication flow, paginated player and team browsing, and collection management are all consumable by a real client with minimal friction.

This also validates the API design itself: endpoints that are awkward to consume from a frontend often indicate design issues, and building even a simple client surfaces these problems in a way that reading the spec alone does not. The frontend is accessible at comp3011.evanmadurai.co.uk.

Model Context Protocol (MCP) Server

The application exposes a Model Context Protocol (MCP) server, allowing Large Language Models (LLMs) to directly query the API as a tool. MCP is an emerging open standard developed by Anthropic that defines a common interface for AI models to interact with external data sources and services.

The MCP server exposes six tools drawn from the public read endpoints of the API:

- `health` — returns the current API status
- `search_players` — searches players by partial name or birth country
- `get_player_stats` — retrieves career batting stats, career highs, and physical attributes for a given player ID
- `get_league_leaders` — returns ranked leaders for a given stat, optionally filtered by year and league
- `list_teams` — lists teams with optional league and year filters
- `get_team_season` — retrieves a team's full season record for a given year

This enables an interesting use case: an LLM can answer natural language baseball queries — *"Who led the AL in home runs in 1927?"* or *"Compare the 2004 Red Sox and Yankees seasons"* — by autonomously calling the appropriate tools and synthesising the results. This extends the utility of the API beyond traditional HTTP clients, positioning it as a data source for AI-powered applications.

Deployment

Many deployment methods were considered, everything from Kubernetes on a bare-metal VPS to integrated cloud solutions. A requirement for me was that this project would be relatively easy to deploy and scale, so I could focus more on features of the web service.

The deployment is done on Render. This was chosen as it was a fast and cost efficient place to deploy a web application with a PostgreSQL instance with good support for Github CI/CD based deployments.

Deploying a new version is done automatically on any push or merge into the main branch of the Github remote repository.

A custom domain (comp3011.evanmadurai.co.uk) was configured via DNS.

Testing Strategy

The testing strategy for this application was inspired by the *testing pyramid* [9], an industry-standard method for designing testing strategies for web applications. The idea is that tests should be layered, with a broad base of fast, isolated unit tests forming the foundation to catch logic errors early. Higher levels of the pyramid consist of integration and end-to-end (E2E) tests, which verify that the NestJS modules, database, and external APIs interact correctly, ensuring the system's overall stability.

For a simple API like this, I found it better to simplify this down to *unit tests* and *E2E tests* as most integration tests end up being E2E tests regardless.

Unit tests

This application has 130 unit tests across 19 suites. These are primarily targeted at testing atomic service functions. These tests were implemented with Jest, and exist in *.spec.ts* files closely coupled with the application code.

These tests provide a code coverage of 78.37%, within the industry standard 75-80% [10]. This implies the tests adequately test the majority of the functionality in the application.

End-to-End tests

The application also has 40 End-to-End (E2E) tests across 5 suites. These test core API functionality from HTTP->Controller->Service.

These tests were implemented using TestContainers, where a real database is provisioned for the tests to execute against. This allows for much closer to production testing than the database mocking traditionally done [11].

Challenges

Configuration Hell

NestJS comes with a learning curve, as does using a new programming language for a new time. However, the main issue was not structurally understanding NestJS nor Typescript, but project configuration.

Setting up a NestJS project using Prisma, Jest, and Typescript was a time-consuming issue, and took a (incredibly frustrating) day or so in it's own right. Had I chosen a more familiar technology stack, this time could have been reduced.

Mocking Prisma

Unit testing NestJS services that depend on Prisma required mocking the Prisma client carefully. This was quite difficult, and while a solution was reached, it requires significant boilerplate for each test suite.

Limitations and areas for improvement

Authentication

A more robust Authentication system would implement use JWT refresh tokens to avoid forcing users to re-authenticate when their access token expires. Token revocation (e.g. on logout) is also not currently supported nor is OAuth2.

Data Staleness

The dataset is a static snapshot imported at deployment time. There is no mechanism to update the underlying Lahman database as new seasons are published. A future improvement could automate periodic re-seeding when new data releases are available, or expose an admin endpoint for bulk data ingestion.

Generative Artificial Intelligence declaration

Generative Artificial Intelligence tools such as Google Gemini, Anthropic's Claude, and Cursor by Anysphere were used in the creation of this coursework.

Idea generation

At the beginning of the project, I provided the coursework brief to Claude Sonnet 4.5 and asked to highlight interesting potential features that could be implemented in my project. It highlighted ideas such as JWT-based authentication, data aggregation endpoints, and rate limiting all of which were later implemented.

Technology stack choice research

Research was conducted by prompting the model for discussion on pros and cons of various technologies, such as whether to use an ORM, GraphQL, or gRPC. Outputs from the model were used to influence my decisions.

Early in the project, I asked Claude Sonnet 4.5 to highlight common Web Frameworks and weigh up the pros and cons of using each. I used this to inform my decision of using NestJS. I also used the model to explore the pros and cons of using an ORM verses direct queries, and it's how I discovered Prisma in the first place.

Implementation

Cursor, an AI-powered integrated development environment, was used in the implementation of this application. While most of the initial application setup was done by hand, after the foundations of the application were set I found that AI agents were extremely useful in speeding up boiler-plate heavy tasks such as creating new endpoints and writing tests. All code was still checked and validated by hand.

In particular, the AI tools were fond of making unit tests that upon closer inspection were so mocked that no real functionality was being tested. I see this kind of plausible-at-a-glance yet completely vacuous code as a potentially dangerous trap.

I am still generally quite averse to getting AI to write english, I find it's outputs information-sparse and grating to read. I find that lots of writers use AI to turn bullet points into full paragraphs, and then lots of readers use AI to turn those paragraphs back into bullet points. Maybe we would just be better use writing bullet points.

On generative artificial intelligence for computer programming

Economists often make the distinction that automation typically automate **tasks**, not **jobs** [12]. In the age of AI, feels more pertinent than ever, my experiences echo that of others [13] who have found that you first must reduce your *job* (writing a REST API) into *tasks* (e.g: writing new endpoints, writing tests, etc). You can then somewhat effectively delegate these tasks to Artificial Intelligence.

For now, my *job* seems safe. It has just taken a different shape, it is starting to be comprised of a different set of tasks than previously. When fully buying into AI powered development cycle I am positioned as more as a programming manager than a programmer - more focused on system design and task delegation, while artificial intelligence tools do the programming…

…how I feel about that is a question for another day.

Bibliography

[1] B. Beighton, "NestJS: The Pros and Cons." [Online]. Available: https://bradbeighton.medium.com/nestjs-the-pros-and-cons-aff714607b07

[2] NestJS Authors, "NestJS - A progressive Node.js framework." [Online]. Available: https://docs.nestjs.com/

[3] Prisma Data Guide, "What is an ORM?." [Online]. Available: https://www.prisma.io/dataguide/types/relational/what-is-an-orm

[4] N. Burk, "Performance Benchmarks: Comparing query latency across TypeScript ORMs and Databases." [Online]. Available: https://www.prisma.io/blog/performance-benchmarks-comparing-query-latency-across-typescript-orms-and-databases

[5] Red Hat, "What is CI/CD?." [Online]. Available: https://www.redhat.com/en/topics/devops/what-is-ci-cd

[6] R. T. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," PhD Dissertation, 2000. [Online]. Available: https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm

[7] NestJS Authors, "Validation - NestJS Techniques." [Online]. Available: https://docs.nestjs.com/techniques/validation

[8] J. Albert and J. Bennett, *Curve Ball: Baseball, Statistics, and the Role of Chance in the Game*. New York: Springer, 2001. [Online]. Available: https://imusic.fr/books/9780387988160/jim-albert-2001-curve-ball-baseball-statistics-and-the-role-of-chance-in-the-game-hardcover-book

[9] J. Schmitt, "The testing pyramid: Strategic software testing for Agile teams." [Online]. Available: https://circleci.com/blog/testing-pyramid/

[10] S. Pittet, "What is code coverage?." [Online]. Available: https://www.atlassian.com/continuous-delivery/software-testing/code-coverage

[11] AtomicJar, "Introducing Testcontainers." [Online]. Available: https://testcontainers.com/guides/introducing-testcontainers/

[12] D. H. Autor, F. Levy, and R. J. Murnane, "The Skill Content of Recent Technological Change: An Empirical Exploration," *The Quarterly Journal of Economics*, vol. 118, no. 4, pp. 1279–1333, 2003, doi: 10.1162/003355303322552810.

[13] C. Towers-Clark, "AI Agents Work When You Automate Tasks, Not Entire Jobs." Accessed: May 22, 2024. [Online]. Available: https://www.forbes.com/sites/charlestowersclark/2025/11/03/ai-agents-work-when-you-automate-tasks-not-entire-jobs/

Appendices

- Generative Artificial Intelligence chat logs
- Web Framework Research

>Weigh up the pros and cons of different typescript web frameworks

>TypeScript Web Frameworks: Trade-offs Compared Here's a breakdown of the major players, grouped by category:

Minimalist / Unopinionated

Express.js

The old guard. Enormous ecosystem, ubiquitous documentation, and almost universal familiarity.

✅ Virtually zero learning curve, massive community ✅ Total flexibility — bring your own structure ❌ TS support is bolted on, not native — types via ❌ No opinions means inconsistent codebases across teams ❌ Middleware model is showing its age; error handling is clunky

Fastify

A modern Express alternative built with performance and TypeScript in mind.

✅ Excellent native TS support with schema-based type inference ✅ Significantly faster than Express (benchmarks consistently show 2–3×) ✅ Built-in JSON schema validation ❌ Smaller ecosystem than Express ❌ Plugin architecture can feel restrictive

Opinionated / Full-Structure

NestJS

The Angular-inspired framework that brings heavy structure to Node backends.

✅ First-class TypeScript — decorators, DI, modules feel natural ✅ Excellent for large teams; enforces consistent architecture ✅ Broad ecosystem: built-in support for GraphQL, WebSockets, microservices, queues ❌ Steep learning curve; significant boilerplate for simple services ❌ Angular-style conventions feel foreign to developers from other backgrounds ❌ Overkill for small services or solo projects

Full-Stack / Meta-Frameworks

Next.js

Primarily a React framework, but its API routes and server actions make it common for full-stack TypeScript services.

✅ Unified codebase for frontend and backend ✅ Excellent DX, Vercel deployment is seamless ✅ Strong community and React ecosystem ❌ Backend capabilities are secondary — not designed for complex API-only services ❌ Opinionated file-based routing can fight you on non-standard patterns ❌ Vendor pressure toward Vercel infrastructure

Remix A newer full-stack framework emphasising web fundamentals and progressive enhancement.

✅ Thoughtful data loading model (loaders/actions) reduces client-side complexity ✅ Strong TS integration; co-locates data fetching with routes cleanly ❌ Smaller community than Next.js; fewer learning resources ❌ Mental model shift required from SPA-style thinking

Edge / Modern Runtimes Hono A lightweight framework built for edge runtimes (Cloudflare Workers, Deno, Bun) but runs anywhere.

✅ Extremely fast and tiny ✅ First-class TypeScript; elegant, chainable API ✅ Runtime-agnostic — works on Node, Bun, Deno, and edge platforms ❌ Young ecosystem; fewer batteries included ❌ Less suitable for complex, stateful backend architectures

tRPC Not a traditional framework, but increasingly central to TypeScript stacks — provides end-to-end type safety between server and client.

✅ Eliminates the client/server type boundary entirely — no codegen needed ✅ Superb DX when paired with Next.js or Remix ❌ Tightly couples frontend and backend; poor fit for public APIs ❌ Not meaningful outside a TypeScript monorepo context

■ ORM Technology Research

> Discuss the pros and cons of using an ORM for web service development

> **ORM Trade-offs in Web Service Development**

An ORM (Object-Relational Mapper) sits between your application code and the database, translating between objects and relational tables. Here's a balanced look:

Pros Developer productivity

Writing user.save() instead of INSERT INTO users … is faster to type, easier to read, and less error-prone. Teams move quicker in early development stages. Database portability

A well-written ORM lets you switch from PostgreSQL to MySQL (or SQLite for tests) with minimal code changes — the ORM handles dialect differences. Security by default

Most ORMs parameterize queries automatically, which largely eliminates SQL injection risks without requiring developer discipline.

Reduced boilerplate

Migrations, schema definitions, and relationship traversal (order.customer.address) are handled without repetitive glue code. Consistency with application code Keeping data access in the same language and paradigm as business logic makes the codebase more cohesive — especially valuable for teams less comfortable with raw SQL.

Cons

The N+1 problem

The classic ORM trap: fetching a list of orders, then lazily loading each customer in a separate query. Without careful use of eager loading, you unknowingly issue hundreds of queries. Performance ceiling ORMs generate generic SQL. For complex analytical queries, bulk operations, or heavily optimised access patterns, hand-written SQL almost always wins. The ORM's abstraction works against you at the edges. Leaky abstractions You still need to understand the SQL being generated. Developers who rely on the ORM without understanding the underlying queries often produce slow, incorrect, or non-scalable code. Impedance mismatch Relational data and object graphs don't map cleanly. Concepts like inheritance, polymorphism, and many-to-many relationships become awkward and often produce convoluted schema designs. Migration complexity As schemas evolve, ORM migration tools can become difficult to manage — especially in large teams where merge conflicts in migration files are common. Vendor lock-in to the ORM itself Switching ORMs (e.g., from Hibernate to jOOQ) can be as painful as switching databases. Your data layer becomes tightly coupled to the ORM's conventions.

When to lean toward an ORM

CRUD-heavy services with straightforward domain models Small teams prioritising development speed Projects where portability across databases matters When reducing SQL injection surface area is a priority

When to lean away

Data-intensive services with complex queries or heavy aggregations High-throughput systems where query control is critical Teams with strong SQL expertise who find the abstraction more hindrance than help Services built around stored procedures or database-specific features

The pragmatic middle ground

Many mature projects use a hybrid approach: an ORM for routine CRUD and relationship management, with raw or query-builder SQL dropped in for performance-critical paths. Tools like SQLAlchemy (Python), jOOQ (Java), or Knex.js (Node) support this blend well. The key is treating the ORM as a tool with a scope, not a universal solution.

■ Idea and work plan generation

> I am completing this coursework. I am using typescript/nestjs/prisma/postgres. I am creating a REST API for baseball data. The seed data is scraped from the lahman dataset. Create a plan for what to implement in a series of steps to get top marks.

> Phase 1: Complete CRUD Operations (CRITICAL - Week 1) Current Status: Only READ implemented ❌ **Step 1.1: Implement CREATE endpoints** POST /teams - Add new team season data POST /players - Register new players POST /batting-stats - Add batting statistics POST /pitching-stats - Add pitching statistics Include proper validation (DTO with class-validator) Return appropriate status codes (201 Created)

Step 1.2: Implement UPDATE endpoints PATCH /teams/:teamID/:yearID - Update team statistics PATCH /players/:playerID - Update player information PUT /players/:playerID/career-stats - Update career stats Implement partial updates vs full replacement logic

Step 1.3: Implement DELETE endpoints DELETE /teams/:teamID/:yearID - Remove team season DELETE /players/:playerID - Remove player (with cascade considerations) DELETE /stats/:statID - Remove specific statistics Return 204 No Content on success

Step 1.4: Add proper error handling 400 Bad Request for validation errors 404 Not Found for missing resources 409 Conflict for duplicate entries 500 Internal Server Error with safe messages

Phase 2: Advanced Features for 70-79+ (Week 2) Step 2.1: Authentication & Authorization Implement JWT-based authentication POST /auth/register - User registration POST /auth/login - User login Protect CREATE/UPDATE/DELETE with Bearer tokens Role-based access (admin vs regular user)

Step 2.2: Advanced Query Features Pagination: GET /players?page=1&limit=20 Filtering: GET /teams?league=AL&year=2024 Sorting: GET /players?sortBy=battingAverage&order=desc Complex queries: GET /players/top-performers?stat=HR&season=2024&limit=10

Phase 3: Excellence Features for 80-89 (Week 3) Step 3.1: Testing Suite Unit tests for services (Jest) Integration tests for endpoints (Supertest) E2E tests for critical user flows Achieve >80% code coverage Document test approach in technical report

Step 3.2: Advanced Architecture Implement CQRS pattern (Command Query Responsibility Segregation) Add Redis caching for frequently accessed data Implement rate limiting (express-rate-limit) Add request logging with Winston Use Swagger/OpenAPI for automatic documentation

Step 3.3: Data Enrichment Add calculated fields (WAR - Wins Above Replacement) Implement sabermetrics calculations (OPS+, ERA+, wRC+) Add player similarity algorithms Historical comparisons and percentile rankings

Step 3.4: External Deployment Deploy to Railway/Render/Fly.io (not just PythonAnywhere) Set up PostgreSQL on cloud platform Configure environment variables properly Add health check endpoint: GET /health

Phase 4: Outstanding Features for 90-100 (Week 4) Step 4.1: Model Context Protocol (MCP) Server

Step 4.2: Novel Data Integration Integrate with Baseball Reference API (if available) Add real-time game scores (ESPN API) Weather data correlation with performance Social media sentiment analysis for player popularity

Step 4.3: Advanced Analytics Endpoints POST /analytics/predict-performance - ML model predictions GET /analytics/trade-suggestions/:teamID - Optimization algorithms GET /analytics/injury-risk/:playerID - Pattern analysis GET /analytics/clutch-performance - Situational statistics

Step 4.5: GraphQL Alternative Endpoint Implement GraphQL alongside REST Allow flexible querying: { player(id: “troutmi01”) { name, stats { HR, AVG } } } Justify in report why both are offered

Phase 5: Documentation & Presentation (Week 5) Step 5.1: API Documentation Generate Swagger/OpenAPI spec automatically Add examples for every endpoint Document authentication flow with diagrams Include curl examples and response schemas Convert to professional PDF

■ Testing suite inspiration

> I am writing a REST API for this coursework in NestJS/Prisma/Postgres

I currently have no testing setup, please outline the easiest way to get satisfactory testing going, outline the kinds of tests, how mocking is relevant, etc. please be concise

Testing in NestJS for this Coursework Easiest setup: Jest (built-in to NestJS) NestJS ships with Jest preconfigured — you likely already have jest in your package.json. Just run npm test.

Two test types you need

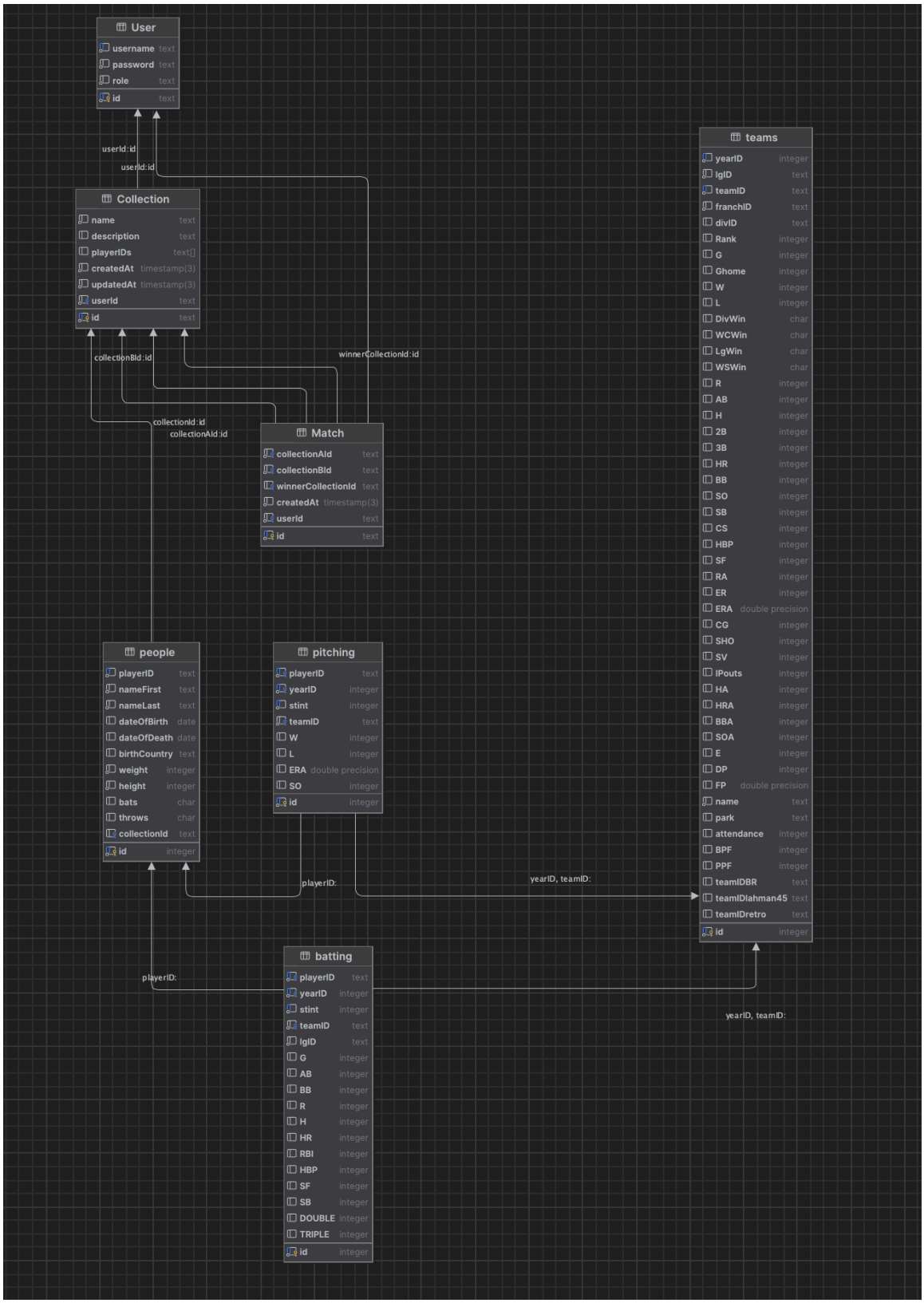
1. Unit tests (most important for the rubric)

Test your service layer in isolation. This is where mocking is essential — you mock Prisma so tests don't need a real database.

2. E2E tests (shows the API actually works end-to-end)

Spins up the full NestJS app and hits real HTTP endpoints, using a test database or mocked Prisma.

Why mocking matters Without mocking, tests need a live Postgres instance. By swapping PrismaService with a jest.fn() object, your unit tests are fast, isolated, and runnable anywhere — including CI. Mock the return values to simulate happy paths and error cases.

Minimum viable test suite for a Good/60+ grade Cover at least one test per CRUD operation in your service, plus one E2E test per major endpoint. That's roughly 8–12 tests total — very achievable in an afternoon. For the 70+ band, add tests for error handling (e.g. 404 when record not found).

Full database ERD